Lab test

Rules, format, two practice specs

Dinu-Ștefan Rusu MEC

Lab objectives

Prove three skills, once

Build a small app from a one-page spec in ninety minutes: a device message, an offline write, and a remote read.

Worth 15 points

5 points per requirement, graded when you present the app at the end of the test.

Time
90 minutes

Present
To the lab instructor, when the time is up

Rules

1. You work alone. No pair programming, no shared screens, no copying a classmate’s code.
2. Ninety minutes, one sitting, starting when you receive the spec.
3. Your own laptop, your existing lab code from Labs 1 to 6.
4. A real board or the Wokwi simulator, whichever you used in the labs. Either is accepted.

The clock does not pause. A crashed simulator or a broker that refuses a connection counts against your ninety minutes.

Allowed resources

1. Internet access is allowed: official docs, package pages, your own past labs.
2. AI agents are allowed for writing code.
3. You must be able to explain any line the grader points to, in your own words.
4. Asking another person, in the room or online, to write code for you is not allowed.

Watch out

An agent-written line you cannot explain earns no credit for that requirement, even if it compiles.

Infrastructure for the test

A broker on the instructor’s machine

A Mosquitto broker for the MQTT requirement. No need to run your own.

Two Go servers on the instructor’s machine

The outbox server from Lab 5 and the gRPC server from Lab 6, already running.

Addresses and ports are announced at the start of the test. Write them down before the clock starts.

Test format

One page, handed at the start

An app idea and three requirements, on paper or a shared document. Nothing changes once the clock starts.

Ninety minutes, then you present

Build until time is up, then run the app for the lab instructor. No extensions.

Every spec asks for the same three kinds of requirement, described next.

Requirement types

Publish or subscribe over MQTT

Talk to the broker: publish a reading, or subscribe and show a value live, with reconnect handled.

One offline-capable write

A local write goes through an outbox and reaches the server once connectivity returns.

One gRPC or BLE read

A unary call to the Go server, or a characteristic read from a peripheral, parsed into a model.

Rubric

Requirement	Full credit (5 p)	Partial credit (2.5 p)
MQTT pub or sub	Connects, and reconnects after a broker drop	Connects once, but does not recover from a drop
Offline write + outbox	Written offline, reaches the server once online	Outbox exists, but the drain loop never runs
gRPC or BLE read	Parses the real response into a model, shown live	Call or read succeeds, but fields come from a raw map

Anything not attempted scores zero.

Presentation

1. Stop coding when the ninety minutes end.
2. Run the app on your emulator or device when you are called.
3. Show each requirement working, in the order of the spec.
4. Answer questions on any line of your code.

Watch out

What runs when you are called is what is graded. The test cannot be presented later.

Preparation

Requirement	Taught in
Publish or subscribe over MQTT	Lab 2
One offline-capable write with an outbox	Lab 5
A gRPC unary call, or a BLE characteristic read	Lab 6, Lab 3

Nothing on the test is new. Re-read the Task slides and the reference code in those three labs. Redo one small piece from memory before you come.

Practice A: greenhouse monitor

An app that shows a greenhouse reading live and logs manual care events.

1. Subscribe with `mqtt_client` to `greenhouse/temp` and show the value live in a BlocBuilder, with reconnect on broker loss.
2. A *Log watering* button writes to a local table and an outbox in one transaction. A drain loop posts it to the outbox server.
3. Call the gRPC server for the care thresholds for the plant and show them once parsed into a model.

Grader checks

- [ ] Stopping and restarting the broker resumes live updates.
- [ ] A watering event logged in airplane mode reaches the server once online.
- [ ] The threshold values on screen come from the model, not a raw map.

Practice B: delivery tracker

An app that tracks a van's status and one delivery scan.

1. Publish the van's status to `fleet/van1/status` every few seconds with QoS 1.
2. Marking a delivery complete writes to a local table and an outbox in one transaction, synced by a drain loop.
3. Connect to the scanner peripheral over BLE and subscribe to the last scanned code, shown live on screen.

Grader checks

- ☐ Publishing resumes after the broker connection drops and comes back.
- ☐ A delivery marked complete offline appears on the server once the sync loop runs.
- ☐ The scanned code updates the screen without polling the peripheral.

Common mistakes

`Client not connected`	You published before the MQTT connect finished. Await the connection state before the first publish or subscribe.
`An outbox row stays queued forever`	The drain loop never marks the row sent after a successful post. Update its status inside the same transaction as the network call.
`A gRPC call hangs and the screen never updates`	No deadline was set. Attach one, and handle the deadline-exceeded status instead of swallowing it.
`BLE scan returns no devices`	A runtime permission was never requested: `BLUETOOTH_SCAN` on Android, the usage string on iOS.

Presentation

Each requirement shown running · presented when time is up